

Cours Développement Web

Niveau Intermédiaire • Atelier – Activité n°1

Interfaces modernes : HTML5/CSS3 avancés & performance (Grid/Flex + API de stockage)

Durée : 04 heures | **Modalité** : individuel | **Remise** : fichier .zip + mini-document explicatif

Objectifs	Renforcer CSS Grid/Flex (options avancées) pour des interfaces modernes et responsives ; appliquer une démarche performance (poids, rendu, animations) ; conserver l'activité API de stockage Web (LocalStorage).
Ressources	Ressources du module + exemples du cours + recherche Internet (si nécessaire).
Type	Activité individuelle.
Outils	VS Code, navigateur moderne, DevTools (Console/Network). Optionnel : Live Server.
Résultat attendu	Un .zip contenant : (A) interface Grid/Flex + (B) LocalStorage v1/v2 + README_Qualite.
Évaluation	Par le tuteur : conformité, qualité UI, responsive, performance, propreté du code et compréhension.

1. Présentation de l'atelier

Cet atelier comporte deux volets. Le Volet A (nouveau) est un entraînement Grid/Flex avancés avec une solution fournie (codes) que vous pouvez copier et tester. Le Volet B conserve l'activité API de stockage Web (LocalStorage) de l'énoncé existant.

2. Partie 0 — Démarrage guidé (environnement, test, mesure)

- Créez un dossier racine : S1_Intermediaire_NOM_Prenom/.
- Créez : 01_GridFlex/ et 02_LocalStorage/.
- Ouvrez le dossier dans VS Code ; ouvrez DevTools (F12).
- Après chaque étape : sauvegarder → rafraîchir → vérifier Console et Network (poids/requêtes).

Arborescence recommandée

```
S1_Intermediaire_NOM_Prenom/  
  01_GridFlex/  
    index.html  
    css/style.css  
    assets/images/ (optionnel)  
  02_LocalStorage/  
    v1/ ...  
    v2/ ...  
  README_Qualite.txt
```

3. Volet A — Atelier Grid/Flex avancés (avec solutions)

Vous allez créer une interface moderne (type dashboard / landing page). Les consignes sont suivies de solutions en code. Vous pouvez soit écrire vous-même, soit copier la solution et la tester, puis la modifier.

3.1 Cahier des charges (obligatoire)

- Une page : 01_GridFlex/index.html + 01_GridFlex/css/style.css.
- Structure : Header (logo + nav), Sidebar (optionnelle), Main (cartes/sections), Footer.
- Header en Flexbox, layout global en Grid (areas ou 12 colonnes).
- Grille de cartes responsive : repeat() + minmax() + auto-fit/auto-fill.
- 2 breakpoints minimum (ex. 900px et 600px).
- Performance : sobriété (≤2 polices), images légères ; animations raisonnables.

3.2 Exercices progressifs (guidés) + solutions

Étape 1 — Squelette HTML sémantique

- Créez index.html avec <header>, <nav>, <main>, <aside> (optionnel) et <footer>.
- Liez la feuille CSS : css/style.css.

Solution (index.html — version de base)

```
<!doctype html>  
<html lang="fr">  
<head>  
  <meta charset="utf-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1">  
  <title>Volet A — Grid/Flex</title>  
  <link rel="stylesheet" href="css/style.css">
```

```

</head>
<body>
  <header class="header">
    <div class="brand">
      <span class="logo" aria-hidden="true">◆</span>
      <span class="brand__name">Mon Interface</span>
    </div>

    <nav class="nav" aria-label="Navigation principale">
      <a href="#overview">Aperçu</a>
      <a href="#cards">Cartes</a>
      <a href="#stats">Statistiques</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <div class="layout">
    <aside class="sidebar" aria-label="Menu secondaire">
      <h2 class="sidebar__title">Menu</h2>
      <ul class="sidebar__list">
        <li><a href="#overview">Tableau de bord</a></li>
        <li><a href="#cards">Éléments</a></li>
        <li><a href="#stats">Rapports</a></li>
      </ul>
    </aside>

    <main class="main">
      <section id="overview" class="panel">
        <h1>Interface moderne (Grid + Flex)</h1>
        <p>Objectif : layout responsive, clair, et performant.</p>
      </section>

      <section id="cards" class="panel">
        <h2>Cartes</h2>
        <div class="cards">
          <article class="card featured">
            <h3>Carte mise en avant</h3>
            <p>Occupe plus d'espace sur desktop.</p>
            <button class="btn">Action</button>
          </article>

          <article class="card"><h3>Carte 1</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 2</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 3</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 4</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 5</h3><p>Contenu...</p></article>
        </div>
      </section>

      <section id="stats" class="panel">
        <h2>Statistiques</h2>
        <div class="stats">
          <div class="stat"><span class="stat__n">24</span><span
class="stat__l">Tâches</span></div>
          <div class="stat"><span class="stat__n">8</span><span
class="stat__l">Projets</span></div>
          <div class="stat"><span class="stat__n">96%</span><span
class="stat__l">Score</span></div>

```

```

        </div>
    </section>

    <section id="contact" class="panel">
        <h2>Contact</h2>
        <p>Section d'exemple pour tester la navigation par ancres.</p>
    </section>
</main>
</div>

<footer class="footer">
    <small>© 2026 — Volet A Grid/Flex</small>
</footer>
</body>
</html>

```

Étape 2 — Layout global en CSS Grid

- Créez style.css et définissez un layout global en Grid via .layout.
- Option recommandée : grid-template-areas.

Solution (CSS — Grid global)

```

/* css/style.css — Étape 2 : base + Grid global */
:root{
    --bg: #0b1220;
    --panel: #0f1a2f;
    --text: #e9eefc;
    --muted: #b6c2e2;
    --primary: #4cc3ff;
    --space-1: 8px;
    --space-2: 16px;
    --space-3: 24px;
    --radius: 12px;
}

*{ box-sizing: border-box; }

body{
    margin: 0;
    font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
    background: var(--bg);
    color: var(--text);
}

/* Layout global */
.layout{
    display: grid;
    grid-template-columns: 260px 1fr;
    gap: var(--space-2);
    padding: var(--space-2);
}

.sidebar{ background: var(--panel); border-radius: var(--radius); padding: var(--space-2); }

```

```
.main{ display: grid; gap: var(--space-2); }
.panel{ background: var(--panel); border-radius: var(--radius); padding: var(--space-2); }

.footer{ padding: var(--space-2); text-align: center; color: var(--muted); }
```

Étape 3 — Navigation avec Flexbox

- Mettez le header en Flex : aligner logo/nom à gauche et menu à droite.
- Ajoutez un style hover + focus visible.

Solution (CSS — Header Flex + liens)

```
/* Étape 3 : Header en Flex */
.header{
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--space-2);
  padding: var(--space-2);
  border-bottom: 1px solid rgba(255,255,255,0.08);
}

.brand{ display: flex; align-items: center; gap: var(--space-1); }
.logo{ color: var(--primary); font-size: 18px; }
.brand__name{ font-weight: 700; letter-spacing: 0.3px; }

.nav{ display: flex; flex-wrap: wrap; gap: 12px; }
.nav a{
  color: var(--muted);
  text-decoration: none;
  padding: 8px 10px;
  border-radius: 10px;
}
.nav a:hover{ background: rgba(76,195,255,0.12); color: var(--text); }
.nav a:focus-visible{ outline: 2px solid var(--primary); outline-offset: 2px; }
```

Étape 4 — Grille de cartes responsive (auto-fit/minmax)

- Créez une grille de cartes en CSS Grid avec repeat(auto-fit, minmax(...)).
- Ajoutez une carte ‘featured’ qui occupe 2 colonnes sur desktop.

Solution (CSS — Cards Grid + featured)

```
/* Étape 4 : Grille de cartes */
.cards{
  display: grid;
  gap: var(--space-2);
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}

.card{
  background: rgba(255,255,255,0.04);
}
```

```

border: 1px solid rgba(255,255,255,0.08);
border-radius: var(--radius);
padding: var(--space-2);
}

.card h3{ margin-top: 0; }

/* Carte mise en avant : 2 colonnes sur grand écran */
@media (min-width: 900px){
  .featured{ grid-column: span 2; }
}

.btn{
  margin-top: 8px;
  padding: 10px 12px;
  border: 0;
  border-radius: 10px;
  background: var(--primary);
  color: #031019;
  font-weight: 700;
  cursor: pointer;
}

```

Étape 5 — Responsive (2 breakpoints minimum)

- À 900px : réduire la sidebar si besoin ; à 600px : passer en une seule colonne.
- Assurez-vous que le menu ne casse pas la mise en page.

Solution (CSS — breakpoints)

```

/* Étape 5 : Responsive */
@media (max-width: 900px){
  .layout{ grid-template-columns: 220px 1fr; }
}

@media (max-width: 600px){
  .layout{ grid-template-columns: 1fr; }
  .sidebar{ order: 2; }
  .main{ order: 1; }
  .nav{ justify-content: flex-start; }
}

```

Étape 6 — Micro-interactions (hover + transition) + reduced motion

- Ajoutez une transition légère sur les cartes et boutons.
- Bonus : respecter prefers-reduced-motion.

Solution (CSS — transitions + prefers-reduced-motion)

```

/* Étape 6 : Micro-interactions */
.card{
  transition: transform 160ms ease, border-color 160ms ease, background 160ms ease;
}

```

```

.card:hover{
  transform: translateY(-2px);
  border-color: rgba(76,195,255,0.35);
  background: rgba(76,195,255,0.06);
}

.btn{
  transition: transform 160ms ease, filter 160ms ease;
}
.btn:hover{ filter: brightness(1.05); transform: translateY(-1px); }
.btn:focus-visible{ outline: 2px solid #ffffff; outline-offset: 2px; }

@media (prefers-reduced-motion: reduce){
  .card, .btn{ transition: none; }
  .card:hover, .btn:hover{ transform: none; }
}

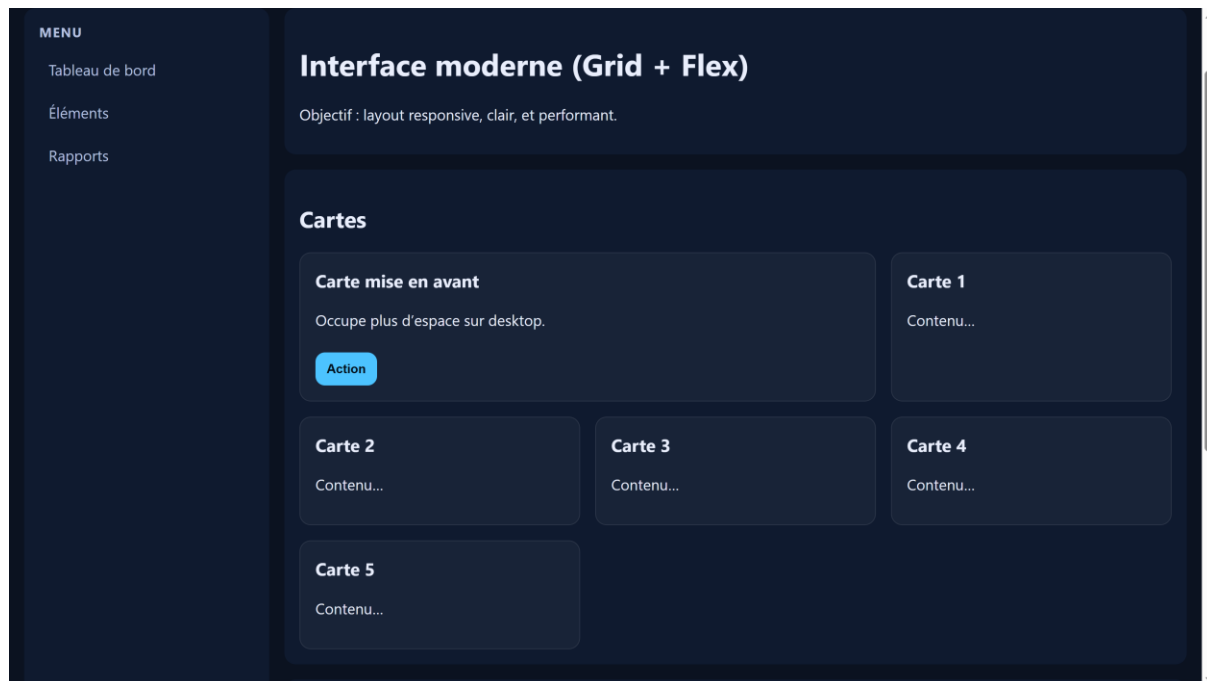
```

Étape 7 — Performance : vérifications rapides

- Ouvrir DevTools → Network : vérifier le poids total et le nombre de requêtes.
- Limiter polices et images ; éviter bibliothèques lourdes ; supprimer CSS inutilisé.
- Noter dans README_Qualite.txt : 2 optimisations faites (avant/après).

3.3 Solution complète du Volet A (à copier/coller)

Ci-dessous, la version complète finale (index.html + style.css). Vous pouvez la copier telle quelle puis l'adapter (couleurs, sections, contenu). Voici le rendu du code ci-dessous.



Fichier : 01_GridFlex/index.html

```

<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Volet A – Grid/Flex</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="header">
    <div class="brand">
      <span class="logo" aria-hidden="true">◆</span>
      <span class="brand__name">Mon Interface</span>
    </div>

    <nav class="nav" aria-label="Navigation principale">
      <a href="#overview">Aperçu</a>
      <a href="#cards">Cartes</a>
      <a href="#stats">Statistiques</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <div class="layout">
    <aside class="sidebar" aria-label="Menu secondaire">
      <h2 class="sidebar__title">Menu</h2>
      <ul class="sidebar__list">
        <li><a href="#overview">Tableau de bord</a></li>
        <li><a href="#cards">Éléments</a></li>
        <li><a href="#stats">Rapports</a></li>
      </ul>
    </aside>

    <main class="main">
      <section id="overview" class="panel">
        <h1>Interface moderne (Grid + Flex)</h1>
        <p>Objectif : layout responsive, clair, et performant.</p>
      </section>

      <section id="cards" class="panel">
        <h2>Cartes</h2>
        <div class="cards">
          <article class="card featured">
            <h3>Carte mise en avant</h3>
            <p>Occupe plus d'espace sur desktop.</p>
            <button class="btn">Action</button>
          </article>

          <article class="card"><h3>Carte 1</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 2</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 3</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 4</h3><p>Contenu...</p></article>
          <article class="card"><h3>Carte 5</h3><p>Contenu...</p></article>
        </div>
      </section>

      <section id="stats" class="panel">
        <h2>Statistiques</h2>

```



```

        <div class="stats">
            <div class="stat"><span class="stat__n">24</span><span
class="stat__l">Tâches</span></div>
            <div class="stat"><span class="stat__n">8</span><span
class="stat__l">Projets</span></div>
            <div class="stat"><span class="stat__n">96%</span><span
class="stat__l">Score</span></div>
        </div>
    </section>

    <section id="contact" class="panel">
        <h2>Contact</h2>
        <p>Section d'exemple pour tester la navigation par ancrés.</p>
    </section>
</main>
</div>

<footer class="footer">
    <small>© 2026 – Volet A Grid/Flex</small>
</footer>
</body>
</html>

```

Fichier : 01_GridFlex/css/style.css

4. Volet B — API de stockage Web (LocalStorage) — activité conservée

Réalisez l'activité LocalStorage (thèmes jour/nuit + persistance) en deux versions (v1 et v2).

- Version 1 : ajouter d'autres thèmes. Livraison : Nom_Prenom_LocalStorage_v1.zip
- Version 2 : ajouter un champ texte + bouton pour mémoriser un paragraphe (thème + paragraphe restaurés). Livraison : Nom_Prenom_LocalStorage_v2.zip.

4.1. Introduction

L'API Web Storage (HTML5) permet de stocker des données localement dans le navigateur, sans base de données externe. Les données sont stockées sous forme de couples clé/valeur (chaînes de caractères). Dans cette annexe, vous allez créer une page qui mémorise le thème choisi (jour/nuit) et, dans une version avancée, mémorise aussi un paragraphe saisi par l'utilisateur.

4.2. Rappels Web Storage

- localStorage : stockage persistant (reste après fermeture du navigateur).
- sessionStorage : stockage limité à la session (supprimé à la fermeture du navigateur).
- Les valeurs sont des chaînes : si vous stockez un nombre/objet, convertissez-le en texte (ex. JSON).

4.3. Vérifier la présence de l'API

Avant d'utiliser le stockage Web, vous pouvez vérifier que le navigateur le supporte :

```
if (typeof(Storage) !== "undefined") {  
    alert("Votre navigateur supporte l'API Web Storage !");  
} else {  
    alert("Votre navigateur ne supporte pas l'API Web Storage.");  
}
```

4.4. Objectif de l'activité pratique

- Créer une page HTML5 avec deux thèmes CSS3 : 'jour' et 'nuit'.
- L'utilisateur choisit le thème via des boutons.
- Le dernier thème choisi est mémorisé dans localStorage et restauré au prochain chargement.
- IMPORTANT (Semaine 1) : ne pas utiliser addEventListener. Utiliser uniquement onload/onclick (événements HTML classiques).

4.5. Wireframing (rappel)

L'interface attendue est simple : un titre, deux boutons (Jour/Nuit) et un paragraphe. Vous pouvez garder une mise en page minimaliste.

Voici les maquettes de l'interface graphique de l'application (dessinées avec Pencil) :



4.6. Étapes de réalisation — Version de base (jour/nuit)

1. Créer le dossier : `activite_localstorage/` avec sous-dossiers `css/` et `js/`.
2. Créer le fichier `activite_localstorage/index.html` et le lier à `css/index.css`.
3. Donner un id à la balise `<body>` (ex. `maPage`) pour pouvoir changer sa classe.
4. Créer deux boutons avec l'attribut `onclick` pour appliquer le thème 'jour' ou 'nuit'.
5. Créer le fichier `js/index.js` : fonctions `appliquerTheme(theme)` et `initialiser()`.
6. Lancer l'initialisation au chargement via l'attribut `onload` dans `<body>`.

4.6.1 Code — index.html (événements classiques)

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Activité Local Storage</title>
  <link rel="stylesheet" href="css/index.css">
</head>

<body id="maPage" onload="initialiser()">
  <h1>API Local Storage – Thèmes</h1>

  <button id="b1" onclick="appliquerTheme('jour')">Thème Jour</button>
  <button id="b2" onclick="appliquerTheme('nuit')">Thème Nuit</button>

  <p id="p1">Ceci est un exemple d'utilisation de l'API Local Storage de HTML5</p>

  <script src="js/index.js"></script>
</body>
</html>
```

4.6.2 Code — css/index.css (définir les thèmes)

Les thèmes sont définis comme des classes appliquées au `<body>` :

```
/* css/index.css */
body {
  font-family: Arial, sans-serif;
  padding: 20px;
}

/* Thème jour */
.jour {
  background: #eeeeee;
  color: #000000;
}

/* Thème nuit */
.nuit {
  background: #000000;
  color: #ffffff;
}
```

```
}  
  
button {  
  margin-right: 10px;  
  padding: 10px 12px;  
  cursor: pointer;  
}
```

Il faut ajouter en plus du fichier HTML et le fichier CSS, le code javascript.

4.7. Travail Personnel — Version 1 (v1)

- Ajouter d'autres thèmes (ex. bleu, vert, contrasté).
- Ajouter un bouton par thème (onclick).
- Mémoriser et restaurer le thème choisi via localStorage.
- Nom du fichier à déposer : Nom_Prenom_LocalStorage_v1.zip

4.8. Travail Personnel — Version 2 (v2) : mémoriser aussi un paragraphe

Dans cette version, l'utilisateur saisit un texte dans un champ, puis clique sur un bouton pour remplacer le paragraphe existant. Le dernier thème ET le dernier paragraphe doivent être restaurés au prochain chargement.

4.8.1 Modifications HTML (ajout d'un champ + bouton onclick)

```
<!-- À ajouter dans index.html (ex. sous les boutons) -->  
  
<input id="txtParag" type="text" placeholder="Saisir un paragraphe...">  
<button onclick="enregistrerParagraphe()">Enregistrer le paragraphe</button>
```

4.8.2 Consignes de remise v2

- Le thème et le paragraphe doivent être persistants après rechargement de la page.
- Le paragraphe par défaut est remplacé par le dernier paragraphe enregistré.
- Nom du fichier à déposer : Nom_Prenom_LocalStorage_v2.zip

5. Démarche Performance & Qualité (obligatoire)

- Documentez dans README_Qualite.txt : 2 choix Grid/Flex + 2 optimisations performance + 1 point accessibilité.
- Vérifiez Console : aucune erreur.
- Vérifiez Network : poids total raisonnable (éviter ressources lourdes).

6. Livrables & dépôt

- Un fichier .zip : NOM_Prenom_S1_Intermediaire_HTML5_CSS3_Avances.zip
- Contenu : 01_GridFlex/ + 02_LocalStorage/ (v1 et v2) + README_Qualite.txt.

7. Critères d'évaluation (barème indicatif /20)

Critère	Attendu	Points
Volet A — Grid/Flex	Layout correct, responsive, options avancées, code CSS propre.	0–7
Volet B — LocalStorage	Fonctionnalités v1/v2, persistance, validations.	0–7
Performance & qualité	Vérifications (Network/Console), optimisations, accessibilité.	0–4
Organisation	Zip structuré, noms corrects, README clair.	0–2